

Asynchronous Messaging in Microservices

Using Async patterns early can boost efficiency and improve customer overall experience.

How to solve common problems in Microservices using Asynchronous patterns

Prerequisites:

Understanding of Microservices architecture

RestFul communications over HTTP

TCP/IP in the context of HTTP

Basic understanding of Asynchronous messaging

When dealing with System to System calls, we have two strategies: *Traditional synchronous model and Asynchronous model for interservices communications*

Standard model of communication (Synchronous):

Service-to-Service communication is over *HTTP using Rest*

ServiceA =====> Request =====> Process request and it responds back to the Service
calls are *synchronous in nature* (meaning the caller service waits for a response and this response is sent after the request is fully processed. Therefore, each call becomes a “*Blocking*” call that the client must wait on a success or failure, based on the status code from the Service being called, before it can proceed with other things.

So call paths can *become deep* and it could become a *bottleneck* for longer processes

Second option is Asynchronous communication:

Two ways to implement Asynchronous communication:

1. HTTP over REST (The client sends a call and the server immediately responds with an accepted status code. Then the client either Polls the server or waits for a Push message to determine if the work was successful and done.)
2. Messaging systems like RabbitMQ, JMS, Kafka etc (there are many different ways to implement messaging patterns like Drop a message and the downstream will work on that message. Success or failure can be communicated in many different ways)

Asynchronous communication is different to get it right.

Benefits of Asynchronous:

1. Offload strain on the system (not every call needs to be a Blocking call)
 2. Improve user experience (not every call needs an immediate attention since there is a correlation between user wait times and satisfaction)
For example, in eCommerce system, these can translate to money
 3. Improve system health
 4. Workflows (Like in DAG workflows, it allows to build natural retries without affecting performance)
- Asynchronous communication can improve system viability, scalability and overall operations

When and why to this Asynchronous communication style?

In TCP/IP communications on a network, you can fail the network because each Node broadcasts its routing table to the whole network (“Broadcast storm”) and the network itself would become gridlocked. A similar pattern can emerge in a microservices architecture. As more and more services enter the system, it can lead to Congestion in the network. TCP itself creates a lot of network load and traffic and many calls added can bring Congestion on a network.

Congestion grows exponentially as more services are added to the network since they all talk to each other over the network (Exponential traffic)

Single slow service can aggravate the condition even more since the latency is felt in all services dependent on this Slow service. If we have Blocking calls, it will even make the latency worse.

Remove synchronous calls

Single most effective way to reduce gridlock is to remove Blocking calls when not necessary. Less traffic on waiting on response can reduce gridlocks.

Long-running processes:

1. Blocking can be an issue on network congestion. Long running processes can aggravate the situation.
2. Asynchronous messaging provides an opportunity to build a natural retry logic without another network call. This helps with system health especially with those long-running processes
3. Long-running processes have usually complex call trees. In Asynchronous communication, call trees are non-existent and it improves performance
4. Reduce risks of Timeouts. Blocking calls in Long-running processes can cause timeouts to clients.

Reduce coupling:

In Synchronous communications, microservices becomes like a Monolith microservices again due to tight coupling. In the end, it becomes only a monolith. In Asynchronous messaging, the consumer is not known so coupling is reduced naturally.

Say one service calls several services, then again services, this call tree can become a major issue. In Asynchronous communication, this doesn't happen.

Additional benefits of Asynchronous:

Quality of Service and priority in messages. This allows consumers to prioritize the work

Fault tolerance (In Synchronous messaging, we wait for response indefinitely if a client is down)

Circuit breaker pattern can be implemented

Logging, metrics, and analytics, metrics/log availability do not have to be immediate so Synchronous is not needed in this case.

Trade-offs:

Complexity:

1. Usually consumers of messages have their own code repositories, build pipelines, deploying pipelines and config management.
2. Disconnected code paths makes debugging, troubleshooting difficult
3. Multiple paths (like fanout design) also increase complexity.

Observability

1. Disconnect in call stacks make Observability harder
2. Lack of immediate response, we can't log errors immediately. We examine different logs to construct a full picture of the process.
3. Difficult to aggregate the logs from different microservices. Usually we add some Correlation IDs to aggregate logs
4. Correlation of metrics also becomes a challenge

Additional complexity due to Asynchronous:

1. Adding more components adds complexity
2. More containers can impact the operational costs
3. Overall operations evaluation becomes a little more tricky
4. Operations should be able to trace its tasks throughout the system without the need for debugging code.
5. Plus expected hops should be documented properly
6. We need to provide much information through Logs for aggregation and quick inspection
7. Operational runbooks are critical
8. Root-cause identification is significantly harder. Tracing issues through call-stacks is harder

Terminologies:

Message broker:

- One of the core components for Asynchronous messaging in microservices architecture.
- Central responsibility for all messaging operations
- It can translate messages from one system into a message for another system. It can translate and transform the messages for the consumers
- Routing: Point to point, inspection-based routing and fanning out messages
- Aggregations: messages can be aggregated as needed. Message brokers can either aggregate or split messages apart.
- Message brokers can handle errors also and respond accordingly

Common message brokers:

RabbitMQ

Apache ActiveMQ

Java Message Service (JMS)

Apache Kafka

Cache like Redis but Caches don't provide the robustness like any other tools

Terms:

Producer is a system that creates the message for some other system. Messages formatted according to the contractual format then dispatched

Consumer or receiver receives messages from Message broker. Once message is received the consumer will act on that message. Consumer also could become another Producer for the downstream processes. After receiving message, it could respond back about the status.

Dead-letter queue is a special place in Message broker where *error messages* go. Several reasons for errors from format issues like structure or length, timeouts from not being read quick enough by the Consumer, queue is full and it cannot accept new messages.

InterService Communication Patterns:

Asynchronous communication or messaging is for Interservice communications. Sometimes, we need to just push a message to a remote system to perform work and you don't want to wait/block on the downstream system finishing its tasks.

1.Point to point messaging:

It is one of the easiest ways in a microservices architecture.

These can replace traditional RestFul calls between services where response calls are not needed.

- Single producer that creates a message and puts on message broker
 - Single consumer responds and listens to the message and performs some actions on it
 - It is a Send and forget model. Producer creates a message, dispatches it and goes on.
- Any kind of response is another point-to-point message.

2.Publish-Subscribe (PubSub):

It is a different model for Interservice communications.

It is a very powerful pattern in distributed computing

- Single producer who creates the message and dispatches it to the message broker
- In this model, there is one or more subscribers (one or more consumers) who listen to the messages. You can have multiple consumers of the same message, it is less common as we don't broadcast the message to many different services with the same exact format.
- Send and forget model (Producer is not waiting on any kind of response before continuing its work)
- Durable subscriptions (In traditional pub-sub if a subscriber isn't there, it won't get the message while if subscriber is durable, it is guaranteed that the message will be delivered at some point once the subscriber is available again)

Point-to-point asynchronous messaging:

When would you use it and how would you construct it?

Use cases:

- 1.When no response is needed from the Consumer (in that case, we can produce the message and no need to wait until the task is done. Audit system is another example where we send a message to the audit system and wait for no response)
- 2.Admin task: Cleanup operations on aggregated data that was denormalized for efficiencies in RDBMS.
- 3.Out of band processes: sending an email, which is asynchronous and doesn't guarantee a delivery. Producer would put on a queue and my communication service will pick it up and send the email.
- 4.Scaling your system: By isolating the jobs, we can scale them individually due to asynchronous messaging. Wait and see system hinders scaling.

Producer =====> Message broker =====> Consumer

Message broker is the heart of the system. Usually the real heavy lifting is in Consumer. Consumer attaches to the Message broker and actively pulls from the Message broker. Consumer goes into the idle state until the next polling time triggers for the message to be listened for. Original service makes a call to the Producer and blocks, waiting for a response from the Producer. Producer creates a message based on the contract and puts it on the Message broker. At this point, the Producer returns as Accepted Status to the original Service. Message broker will present the Message. Consumer in the next listen phase based on polling will pull the Message from Message broker and send Ack to Message broker.

Point to point Considerations:

- 1.Wire time: If the act of building the message, setting up connection to the Message broker, sending a Message to Message broker takes as long as or more than just making the downstream call, are we really saving time?
- 2.Extra components: if complexity of adding more components reduces processing time then it is ok
- 3.Plan for failure scenarios: Use the Dead Letter Queue (DLQ). Inspect DLQ and ensure we are not getting (failure) messages there.

Publish-Subscribe asynchronous messaging:

Use cases

Multiple responders: In Distributed computing, between data centers, you can have triggered actions within each data center to occur in isolation. Those triggers are often admin tasks like triggers to clean up or update search indices. In Pub/Sub model, the central hub can publish the message and a worker in each data center can subscribe to the message. When a new data center comes online, there is no code changes needed, a new data center creates a new Consumer and subscribes to get the messages every other data center gets.

Multiple tasks: when a Consumer asked for a copy of his/her data or asked to be forgotten, we would publish a message to the Message broker. Each department would then listen for those messages and perform their own series of actions to handle the consumer's request. When they were done, they would put a Point-to-point message on a queue and we would then wait until everyone else responded before moving on to the next step. No code changes on the Pub/Sub side, we continued messages putting on the queue.

Consumer choice: The fanout works similar to Pub/Sub. The key here is, you can create messages that everyone or no one will listen to without any code changes. When there is more consumers, they will just subscribe.

Durable subscribers:

- 1.Always get the message:

The Message broker will learn about the Durable subscriber and keep the message around until that Subscriber acknowledges it without impacting other Subscribers.

- 2.This operation becomes Powerful in Mission-critical operations:

Lets say we have a legal obligation to process the data but at the same time don't want to Block or Retry if remote system was down for some reason. Simply we can have the Message brokers to send messages when the Consumers come back online.

3.Unregister if needed:

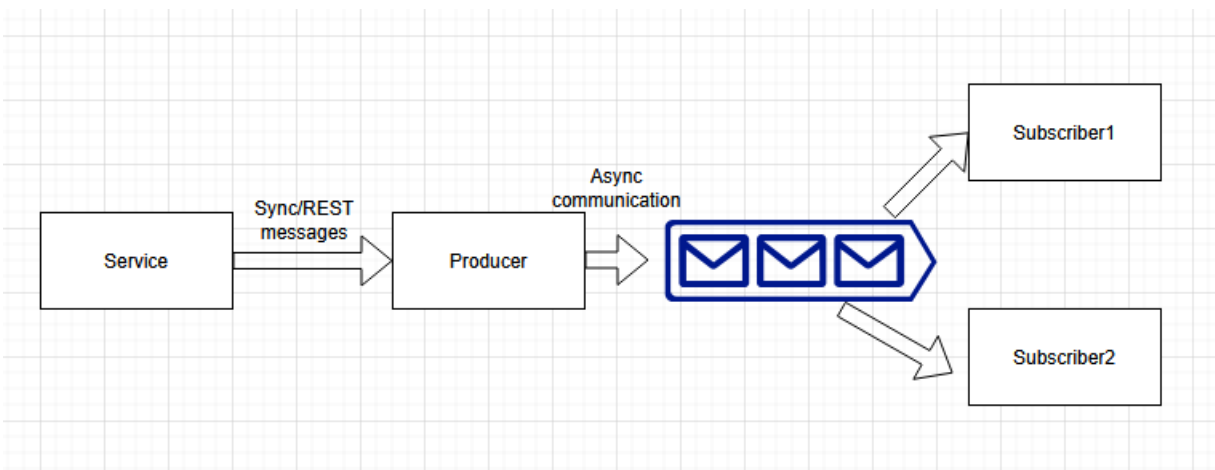
If Consumer no longer needs to consumer the messages, they must unregister to ensure the Message broker doesn't hang on to the messages that are no longer needed.

4.Producer agnostic:

The Producer doesn't need to know the Consumers (not even durable or not). They are completely decoupled.

Publish-Subscribe model

Producer ==> Message broker ==> Subscriber1, and Subscriber2



Producer will be called Synchronously by some Service using RestFul messages. The RestFul messages will come in and start the process. Say we have two Subscribers in the downstream process to act on the message. Each of the Subscribers, subscribes to the messages and starts the pulling process. Message broker also knows currently there are two subscribers for the messages. The Producer creates a message based on the contract and puts on the broker. Then Producer responds back to the original caller service with an Accepted Status code. Message will move to the Message broker and be put on the appropriate queue. It will present message to all available Subscribers. The Subscribers on their own independent listen phase will receive the message in original format. Each subscriber should act on the message on their own while Message broker holds the message.

Event-Driven Microservices:

- Series of steps (Event-Driven Microservices pattern requires one or more steps based on invocation from a specific event. It could be as simple as writing a tuple to a database, or as complex as a series of remediation processes that are required when a specific system event occurs)
- Those steps are triggered from a single event (usually one single event, not like multiple events)
- Toward a same end goal (All steps are part of the same goal. Consider a bank loan application, several tasks or steps must be completed once a loan application is submitted, these events cascade down a path of getting to one end goal – either approval or disapproval of the loan application)
- Each step is distinct and required to reach the end goal. Isolation and concrete definition of steps allows developers to build systems that function as expected.

Choreographed Events:

- Call tree
- Step to step (each step does some work and passes a message down the chain)
- No centralized controller of the choreographed events, they just cascade down the pipeline
- Pipe syntax (in a pipe operator, output of one step is passed onto another). Some trigger and data is passed to the next step.

Orchestrated Events:

- Much more common in practice than Choreographed events
- There is Centralized command and controller

We poll for the result, either from the service executing the step or from a database or some other state store. The orchestrator knows which events are going to be fired and handles the calling downstream services via a message system

- Still based on isolated steps.
- Isolated steps are distinct and has a job to performance

Choreographed events:

much like a dance choreography

Choreographer is the initializer trigger event. Then each step is on its own from there onwards.

Use-cases:

1. Distinct systems:

Initial event execution will trigger the remote system to start its own event chain. The events may trigger other systems. In the context of async messaging and choreographed events, distinct systems are independent microservices or applications that communicate through shared events rather than direct API calls. Instead of one central system "orchestrating" every step, each system acts autonomously, listening for events it cares about and emitting new ones when it completes a task.

2. Alternative cascades:

Each step may have results that trigger more than one next step. As such, the choreography sets the wheels in motion. In a choreographed event-driven architecture, an alternative cascade is an asynchronous fallback mechanism. If a primary event-driven workflow fails or is rejected, the system

triggers a secondary, alternative sequence of events to gracefully resolve the issue, bypassing the need for a central orchestrator.

Event Producer ==> Message broker ==> Step1, Step2, Step3 all unique

Event Producer choreographs the events. In this case, the choreographed event has 3 unique steps. Message is sent to the broker, message is read by Step1 and work is done. New message is sent back to the broker, that new message is read by Step2. Step2 executes then returns another new message. Step3 then reads the message and executes on it. Event Producer is not involved at all after the initial production of the message.

Benefits and trade-offs of this message:

- Increased performance over orchestration (no centralized orchestrator no bottleneck, we offload to the message broker instead)
- Reduced total cost of ownership
- Reduced reliability (since no central place to handle errors, there is more chance that a single event will fail to fire everywhere it is needed. We need to ensure error tracking and DLQs exist in the message broker to handle errors in workflow)
- Reduced observability (since there is no central orchestrator, we don't have a central point of observability. So understanding the status of the event is much more difficult, you got to look everywhere until you find the state of the current message. Also we got to look into message broker and DLQ)

Orchestrated events:

Common pattern within the asynchronous, event-driven microservices model

Use-cases:

1.Sequential processing:

Like Loan application to a Registration event, there is a sequence to follow, you cannot approve a loan before the credit report is returned. Therefore, the command and control center or central orchestrator must ensure it has a response on the credit score before triggering downstream actions. In this model, all communications is asynchronous but the producer of the original message becomes a consumer on the response.

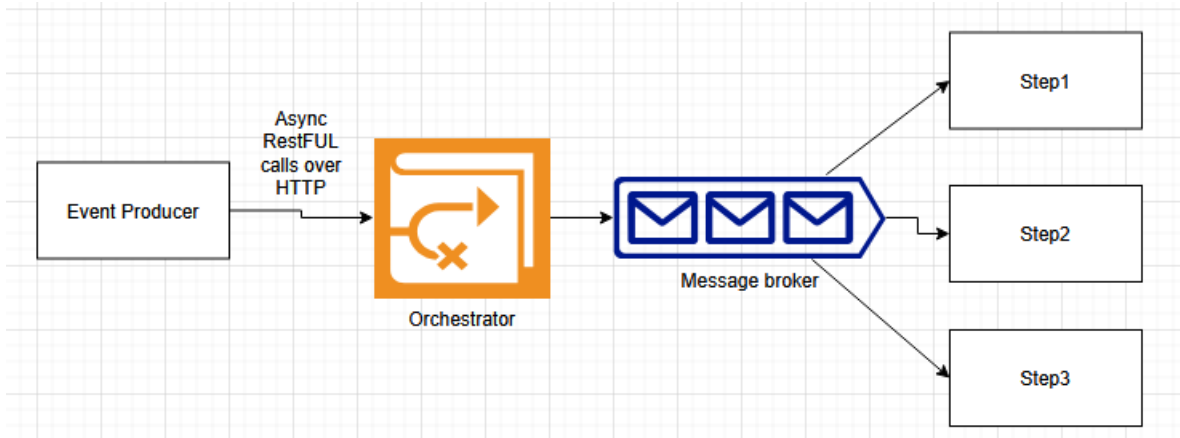
2.Command workflow scenario:

There is a need to allocate infrastructure, before installing Kubernetes into a cloud provider in an automated fashion. You can make direct calls, but the blocking nature of those calls hinders the throughput of the system. So you can dispatch the event to a worker process. The worker does its work, say provisioning virtual machines, when it is done, it sends response message back to the intended command and control center. Once it has the infrastructure it needs, it can then dispatch another message to install Kubernetes to another worker providing the connection details of the newly created virtual machines or databases. This models allows for many Kubernetes clusters to be spun up at once through a single orchestrator call, since the calls become non-blocking asynchronous messages.

3.Response required:

Anytime, you are doing asynchronous messaging and you need a response, be it for sequential processing, command workflows or other use cases, the orchestrator can be the point where this response structure exists. It will aggregate all the responses into a master status that's available to any system interested.

We have an Event producer, which will make RestFul Async calls over HTTP. We then have several steps, each of them knows about the message broker. Event broker makes a call to the Orchestrator, it creates a message and sends it to the broker. The broker exposes the message to Step1 and executes work. Step1 sends message back to the message broker then to Orchestrator. The Poller() from the Event producers polls orchestrator for a response. This process continues until all of our work is done. When finished, the polling will succeed and the entire work is done for our system.



Benefits and trade-offs of this model:

-Decreased performance:

Due to single orchestrator, the steps funnel through a single process (bottleneck) so performance is decreased. Orchestrator must have a state machine of some kind to keep track of the status responses and code paths that handle the downstream actions. This all increases the total cost of ownership

-Increased total cost of ownership

-Increased reliability (in case of failures, the orchestrator can resubmit jobs into processing. You can also build much more robust error handling for outlier conditions)

-Increased observability (Centralized controller becomes the sweet-spot for observability that can help the Operations team)

Hybrid events:

-You still have a centralized command and control center for the system as a whole but based on needs, we can still dispatch choreographed events to remote systems

-Remote choreography (Remote choreographed events can inturn either stay choreographed or convert back to an internal command and control center)

-Converted back to command and control if needed

-Work gets done if the original command and control knows when everything is finished either directly in line or via some other polling mechanism

In Hybrid, the Central Orchestrator directs all traffic internally to the system of record but also kicks off choreographed workloads in remote systems. In the end, the remote system is responsible to alert the centralized command and control center that is it finished

Contract requirements:

- Message contract / format must be well-documented, no room for error
- Contract must be passive to change. When changes happen, they must not break downstream or upstream processes.
- Contracts must be enforced rigorously.

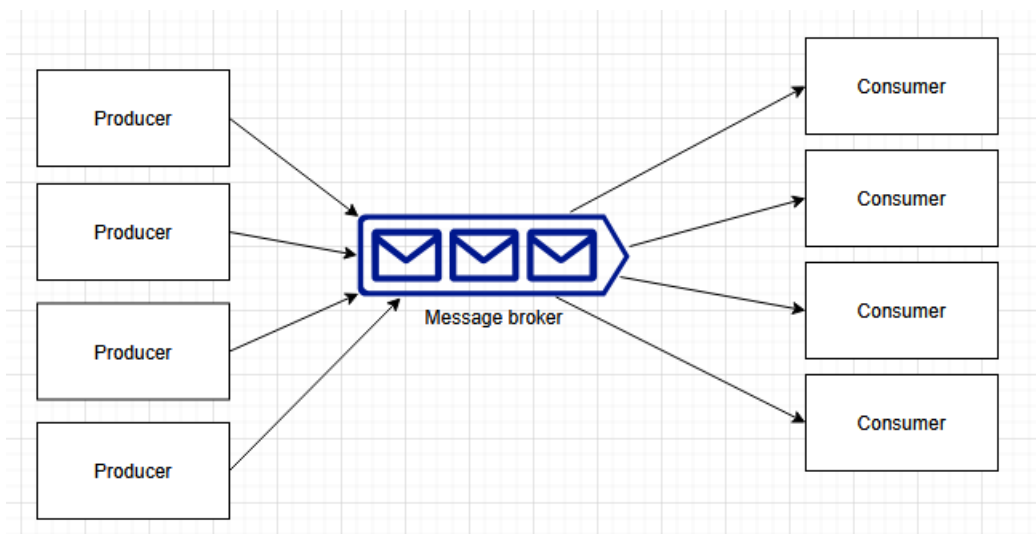
Stream Data Platform

It handles streams of data, mostly from structured log messages that capture every action, every event on the system. So it requires increased disk space on the system, its activity is all handled asynchronously. Very useful in microservices, increased complexity for a reason.

Common Architecture:

Usually a persistent Message broker for the system like Apache Kafka

You have a set of Producers and Consumers and they all are aware of the Message broker. It is a Pub/Sub model with multiple publishers on the same set of topics.



Who are the data Producers:

- 1.Applications produce data/logs. Usually Application itself doesn't transmit the logs/messages into Message broker, instead they only produce them then intermediary like Filebeat shifts log messages.
- 2.Databases also produce logs and events.
- 3.Servers also produce logs, often tell a complete story of what's going on. (Say an Application being compromised, the server logs often have the critical information, really everything can be a Producer in this model if it creates events or logs)

Who are the data Consumers?

1. Log aggregators can help paint a pic of what's really going on in the system. Correlation or trace IDs help the log aggregators to assemble full picture of events in the system by linking the logs together. Also these help maintain a common logging format across different systems through transformations.
2. Analytics engines (since logs cant have a full picture of what's going on), data scientists use this information for long-term and short-term actions
3. Long-term storage vehicles are another model
4. Many use-cases in Big-data drive their data flow from a stream data platform. Data can be collected and shipped to data lake for historical analysis
5. Eventing engines (Analytics can inform us about standard usage of the system and if a rogue user abuses the system then it is flagged as an outlier and importantly it can trigger an event to trace the user). It can do more than intrusion detection.

Why we need Stream data platform?

- Every business needs to collect data
- Business drives off data (internal data helps with procurement and resource allocation, External or user data can help drive decisions)
- If we know historical data, we could rollback after a bad decision. It's all about enabling good decision-making

Log Aggregations

1. Message broker (it can aggregate logs from various systems usually in the raw format)
2. Transformations (it is preferred to keep all logs in the original format so as not to lose context or metadata from the log itself).
3. Readability (Readability of logs directly impacts the utilization of them. If logs are disjointed, then it leads to poor readability)
4. Consumption engine (Log messages are transformed into a common format and shipped to a system designed to provide insight and readability into the log messages. A tool like Elastic ELK stack is a great place to store outcomes of these messages)

Stream processors:

- can act as internal log aggregators
- they are used to perform some API calls to perform some action through synchronous events
- they can also be used to trigger events when needed
- They can also create events through asynchronous messaging. These events can trigger various functions throughout the system

Visualization:

Visualization improves human readability of log messages because we have both the message and metadata about it. Ability to filter out metadata and only visualize the log message becomes a huge improvement on human comprehension. Metadata helps understand the real state of the system was in when the log was produced.

Operations can leverage the visualization of logs when troubleshooting as well as inspecting system health.

Logs can also help with debugging especially in a microservices architecture. Calls are disjointed by very nature, by aggregating you get a full picture of the system as a whole. Logs can help by producing a simpler path so you can target the impacted system.

Visualization aids in the analysis of the logs.

Refactoring: Logs help to see if there are negative impacts of refactoring since stream reads from all environments not just production

System analytics:

Use-case

Analytical exercises on the stream data platform. This is after any internal aggregations of the messages. Stream data platform is an optimal place for data analytics due to its speed, efficiency and near real-time activity. It allows data to be analyzed quickly. In an in-process solution you would be restricted to the data available to the individual microservice but in a stream data analytics solution you would have multiple microservices in systems to capture and analyze data from. It happens in near real-time. If you are building analytics for system events, this becomes critical. One of the exciting uses of the platform is analyzing data in-stream using tools like Apache Spark that are designed to handle the size and scope of data possible, data is analyzed quicker. Real-time analytics engine are much quicker in handling disjointed data than handing via a database.

How?

- Write the job
- Execute the job
- Read the result
- Repeat as necessary (iteration)
- Ship the results to data lake

Lot of work involved in building analytic system on top of stream data platform:

- To really achieve optimal results, you need to commit to it and allow it to mature
- Insights are powerful tools
- Your organization must dedicate real efforts to making this work
- Team needs to buy into value
- ROI takes time

Event detection

Eventing use cases

There is always a need to react to real-time behavior of the system

-Stop bad behavior of the system (like operational runtime issues to bad actor attacking systems.

Patterns were identified through analytics, then created real-time analytics jobs within the stream gate of platform when bad actor events occurred, this created a system event that alerted the security operation center)

-Use Event data to trigger positive reactions in the system. Say two discount codes were offered to the customers, They built an engine that evaluated the engagement rate on the two codes for analytics purposes. If they had been using an Event engine, they could have triggered a reaction based on this data. They found 10% discount rate had better engagement rate than the 15% rate.

-Immediate actions and reactions

The key is all of this analytics data out of a stream data platform can actually be used to trigger real-time events, also for the system to heal itself.

Data

Distributed data

Data flows can range from a distributed data and eventual consistency to CQRS-based data rights.

Why?

-Data is slow, (reading, writing data and finding data are all relatively slow processes. There are remote network calls hidden underneath a network call to a data service. There is file IO that usually occurs or large RAM is being utilized to store the database in memory, access issues if we have large number of data)

-Despite data being slow, it is critical to operation of any system.

-Data is growing everyday (for example, a simple aircraft flight records and offloads copious amount of data on the performance and maintenance of the airplane. When data storage increases, it slows down writing and retrieval of data)

Big areas of data that can utilize asynchronous messaging to improve performance:

-Distributed data

-CQRS and what means for microservices architecture

-Data migrations (we move data from one data store to one or more new ones or vice-versa.

Asynchronous patterns thrive here)

-Data synchronization (Data synchronization between systems)

Eventual consistency:

Eventual consistency is a paramount concept in distributed data

ACID and BASE:

ACID allows transactional writes and consistent reads in a database. ACID is a problem in a distributed microservices world.

Atomic transactions (meaning data is all written or isn't)

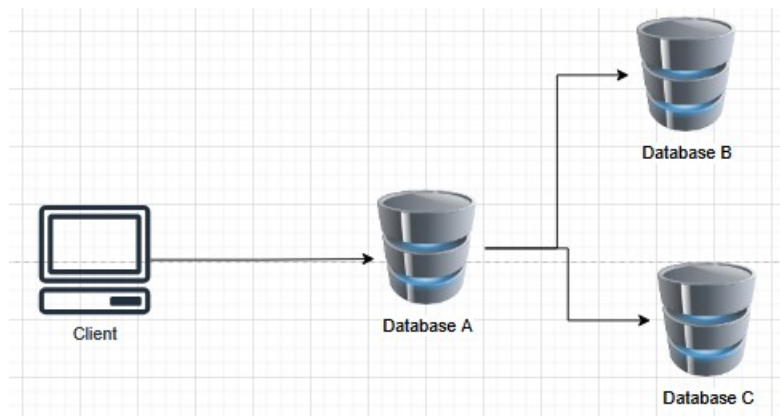
BASE is a different way of thinking. We only care about data is going to be there when we need it. This is E in BASE, Eventual consistency. BA is Basically Available, S is Soft state, meaning data shifts and is eventually consistent. It is always soft and malleable and not Durable like ACID.

Eventual consistency model allows speed up writes to a database with trust the technology will distribute the data to wherever it is needed. Eventual consistency is very important in distributed computing since an Atomic write may timeout the client calling the service.

Eventual consistency like Apache Cassandra

High level

These databases A, B and C are geographically distributed across a region or the entire globe. These databases know about each other and in this example, all have the same data. We have a client that connects to its regional database A for information. When it writes data, it sends to the local database and gets a response back. At this point, other two instances B, C don't have the data, only A does. Within A, there may be internal replication of the data. In Apache Cassandra, you can use quorum to determine how many nodes of the ring need to have the data before the acknowledgment is even sent. Client after sending the message to A, continues with its work. Now A will use messaging of some form to replay the state change to the other two databases. At some point, they will become consistent.



Trade-offs

- Latent reads are possible in this model. In the context of eventual consistency, latent reads (often referred to as "stale reads") occur when a client requests data from a database and receives an outdated version. This happens because the system's most recent write hasn't fully propagated ("eventually consistently") across all its distributed replicas. If you write data, then immediately read it, if you hit a new node in your cluster or global distribution, you may not have the data available you just wrote.
- Communication faults: if communication between Distributed systems fail, the asynchronous communications can also fail. You may need to inject configuration that allows for a fallback read from another instance of a database or load balance globally to prevent issues
- Catastrophic failures: you could write data through a data service and the entire cluster goes down before distributing the data to other instances. That data may not be available anywhere atleast. Circuit breaker patterns can often help in these scenarios.

CQRS:

CQRS is a software pattern for microservices architecture to improve performance

In Microservices architecture, we usually build data services that have specific functions, they usually operate on a single data domain

- Domain data (they usually operate on a single data domain.)
- CRUD operations
- Writes are expensive (CRUD could become expensive over time in traditional RDBMS especially)
- Reads are not immediate or cheap (Reads are not cheap especially when we need to massage the data, if we find a domain whose data services need data being read diverging from the data being written, CQRS may be of help. CQRS (Command Query Responsibility Segregation) helps when your application reads data differently than it writes data. In traditional systems, the exact same database structure and data model are used to both save (write) and view (read) information. However, complex systems often have contrasting requirements for these two actions.)

CQRS is a model that describes written data in a different model than read data.

Command Query Responsibility Segregation

It can be achieved through an asynchronous event-driven architecture

It tends to be focused on Microservices architecture, they are not restricted to microservices

Not a replacement where CRUD works

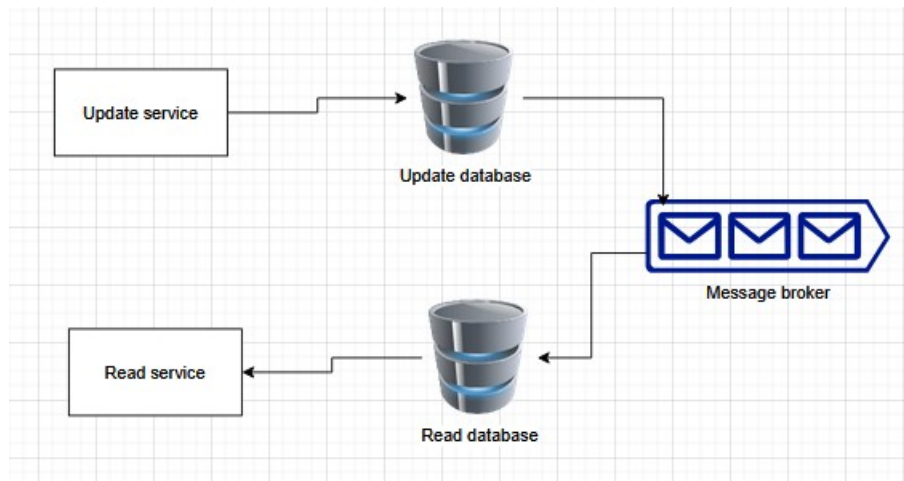
CQRS in Action

Update service in Database, this service uses a distinct domain model that fits in well with its database Likewise, we have a Read service and its database. Again the distinct domain model is different from the update service

Message broker is there, the Producer and Consumer are either in the database itself or via some triggered or event driven process.

Flow: Data is written from the Update service to the Update database and acknowledged. Update service continues with other things. At the same time, message is generated for the event and passed directly to the message broker or to the Read database asynchronously. The message is read by a consumer if the message broker is used and written to the Read database in its transformed state. This consumer may need to read other data to achieve this new model. Now the Read service will hit the

Read database and get a response in the new model. In the context of CQRS (Command Query Responsibility Segregation), a Read Model (or "new model") is a highly optimized, denormalized data structure specifically designed for display and querying, whereas a Write Model focuses entirely on enforcing business rules and transactional integrity for updates.



Write model

```
@Entity
@Table(name = "orders")
public class OrderWriteEntity {

    @Id
    private String id;

    @Column(name = "customer_id", nullable = false)
    private String customerId;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    // Relational mapping used to maintain relational integrity
    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id")
    private List<OrderItemEntity> items;

    // Business methods and logic sit here...
}
```

Read model

```
// A Java Record acts as a high-performance DTO projection
public record OrderReadProjection(
    String id,
    String customerName, // Fetched via a JOIN in the repository query
```

```
double totalAmount,  
String status  
) {}
```

This is a great example for how asynchronous messaging can help solve real-world problems

Data migration

Data migrations are not easy, moving data at their core.

You are usually taking data from all-encompassing database and moving it to a new focused database.

This is often a core part of a microservices migration.

Transforming-Migration isn't just about moving data, it's also about transforming it.

Downtime considerations- we build migration plan accordingly

Orchestration- it is used for its reliability and the ease of use in an event-driven system, like a database write.

Considerations:

Writes are critical to the migration plan and need to be captured

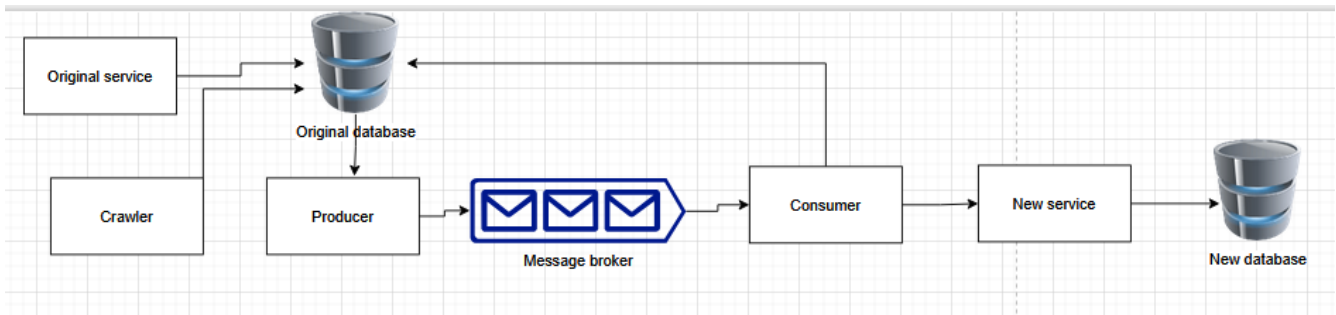
We got to make sure Reads are not impactful to the data.

Transformation impact may have a bigger impact than you expect. Your writes may span across multiple components as well.

Migration time

Model

We start with original system and its database, reads and writes are currently occurring here. You then create a new database and if possible, the new service. Having a new service in place will actually aid in migration and the cut over. Then bring in Message broker and Consumer of that broker. The Consumer should utilize the new service to perform its work if at all possible. The Consumer knows how to read the old database and write to the new database either directly or through the New service. You define the Producer, in this case, a trigger from the database. The Original service sends a create, update or delete message to the Original database then a trigger will fire a message to the Producer, providing the identifier and the action. The Producer will then put a message on the Message broker (queue) for the Consumer. The Consumer will receive the message, the Consumer will then query the Original database and reads the data it needs. Once Consumer has its data, it will call the New service, which in turn writes the action to the New database. You could have a Crawler to crawl the Original database for records that may not be touched.



Data synchronization

Why would we need to synchronize data?

Use-cases

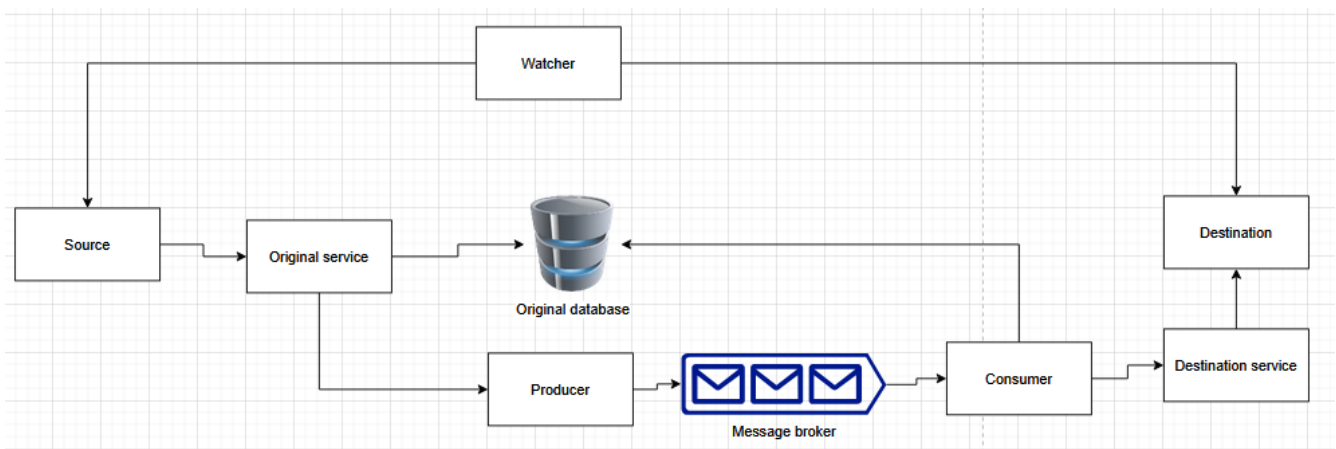
-When we have different database (Runtime database and Data warehouse need to be in sync, but BI system doesn't need to be synchronous to get its data)

-Different systems

We got to build a system to keep data in sync and also ensure they stay in-sync.

Model

Source system is a database, Destination could be a database, cache or search engine. Then we add the Message broker. A trigger from Source system puts message on Broker via a Producer. A Consumer picks up the message either reads it from Source or just processes the message depending on the completeness. Often we will post the ID only and allow Consumer to query the database. It pushes the message to the destination usually via an API. You need to make sure all data is indeed in sync. We introduce Watcher, its purpose is to routinely evaluate data in both systems to ensure that they are in sync. Mostly it will be in sync, in case of discrepancy, it needs to trust the Source system / database and push a message to the Producer to update the Destination



Summary

Asynchronous messaging

Interservice communication patterns like Point to point and Pub/Sub models

Event-driven systems using Asynchronous messaging

Stream data platform

Data use cases that benefit from asynchronous platform